Team Name : india-runs-data-ai

Team Leader Name : Mohamed Fasidh

Problem Statement : Build an offline AI system that ranks candidates for Redrob's Senior AI Engineer founding team role by understanding production ML fit, behavioral availability, logistics, and profile consistency beyond keyword matching.

Solution Overview

- Proposed solution: an offline deterministic ranker that streams the 100K candidate JSONL, extracts JD-aligned signals, scores every candidate, and writes a validated top-100 CSV.
- It is designed for the exact Redrob JD: Senior AI Engineer with production retrieval, ranking, vector search,evaluation, Python, product-engineering, and India/Pune/Noida fit.
- Differentiator: the ranker does not count AI keywords. It combines career evidence, skill trust, Redrob behavior, logistics, education, and honeypot/profile consistency checks.
- Every selected candidate receives a grounded 1-2 sentence reason using only facts present in the candidate profile.

JD Understanding & Candidate Evaluation

- JD requirements extracted: 5-9 years preferred, production embeddings/retrieval, vector or hybrid search,ranking evaluation (NDCG/MRR/MAP/A-B tests), strong Python, and product-company shipping judgment.
- Positive career signals: applied ML/AI/search/recommendation titles, shipped production ranking/search systems,product or marketplace company exposure, recent hands-on coding, and mentoring ability.
- Negative fit signals: pure research without deployment, recent-only LangChain demos, services-only career history, non-technical current titles with AI keyword stuffing, and weak NLP/IR exposure.
- Behavioral signals: open_to_work, recent activity, recruiter response rate, notice period, interview completion,GitHub activity, verification, recruiter saves, and relocation/hybrid readiness.

Ranking Methodology

- Retrieval/ranking: all candidates are scored in one streaming pass, then sorted by score and candidate_id for deterministic output.
- Skill component: Python, retrieval/RAG, vector databases, ranking/recommendations, evaluation, LLM/fine-tuning,and production system evidence with endorsement/duration/assessment trust.
- Career component: current title, experience band, role descriptions, product-company exposure, services-only penalties, shipped search/ranking/recommendation evidence, and research/demo-only penalties.
- Final score: weighted combination of skill (30%), career (34%), behavior (16%), logistics (12%), education (8%),minus honeypot penalties.

Explainability & Data Validation

- Explanations are generated from candidate fields already loaded by the scorer: title, years, location, matched skill groups, top profile skills, response rate, notice period, and open_to_work.
- No hosted LLM is used for reasoning, so justifications cannot invent employers, skills, or achievements outside the profile.
- The tone changes by rank bucket: strong fit, good fit, or borderline fit. Obvious concerns such as long notice,inactivity, weak relocation, or consistency flags are included.
- Suspicious profiles are downweighted for expert skills with near-zero duration, broad skill lists with low experience, AI keyword-heavy non-technical titles, and timeline inconsistency

End-to-End Workflow

1. Read the Senior AI Engineer JD and convert it into explicit feature groups.
2. Stream each candidate JSON object from candidates.jsonl or candidates.jsonl.gz.
3. Build a profile text blob from profile, career history, skills, education, certifications, and Redrob signals.
4. Score skills, career, behavior, logistics, education, and honeypot risk.
5. Sort candidates by final score and deterministic candidate_id tie-break.
6. Write candidate_id, rank, score, and grounded reasoning to submission.csv.
7. Validate using the official validate_submission.py script.

System Architecture

- Input Layer: candidates.jsonl + Redrob JD + candidate schema.
- Feature Layer: text evidence, skill trust, career history, Redrob behavior, logistics, education, and consistency checks.
- Scoring Layer: weighted interpretable components and honeypot penalties.
- Ranking Layer: deterministic sort, strictly decreasing displayed scores, top-100 selection.
- Output Layer: validated submission.csv, metadata YAML, README, GitHub repo, and Colab sandbox demo.

Results & Performance

- Generated the required 100-row submission.csv and passed the official validator: 'Submission is valid'.
- Full 100K-candidate run completed locally in about 54-100 seconds, under the 5 minute CPU-only challenge limit.
- Memory footprint is small: the script streams input and stores compact score rows before sorting.
- Ranking quality is driven by JD-specific evidence rather than generic similarity: top candidates show retrieval/ranking/vector/evaluation production signals plus availability.
- Colab sandbox demo runs successfully on demo_candidates.jsonl and prints 'Demo ranking generated successfully.'

Technologies Used

- Python standard library for ranking: json, csv, gzip, argparse, datetime, pathlib, math, and regex.
- No external model, GPU, or network dependency during ranking, matching the reproduction constraints.
- Pandas is used only in the Colab demo to display the generated demo_submission.csv.
- Matplotlib/lxml are used only for documentation/deck generation, not for ranking.
- GitHub hosts source code and artifacts; Google Colab provides the sandbox/demo notebook.

Submission Assets

- GitHub repository: https://github.com/Mohamed-Fasidh/redrob-candidate-ranker
- Sandbox/demo:https://colab.research.google.com/github/Mohamed-Fasidh/redrob-candidate-ranker/blob/main/redrob_ranker_demo_colab.ipynb
- Submission CSV: submission.csv (validated with official script)
- Reproduce command: python rank.py –candidates ./candidates.jsonl –out ./submission.csv
- Contact: Mohamed Fasidh | mohamedfasidh045@gmail.com | +91-8015334576

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU